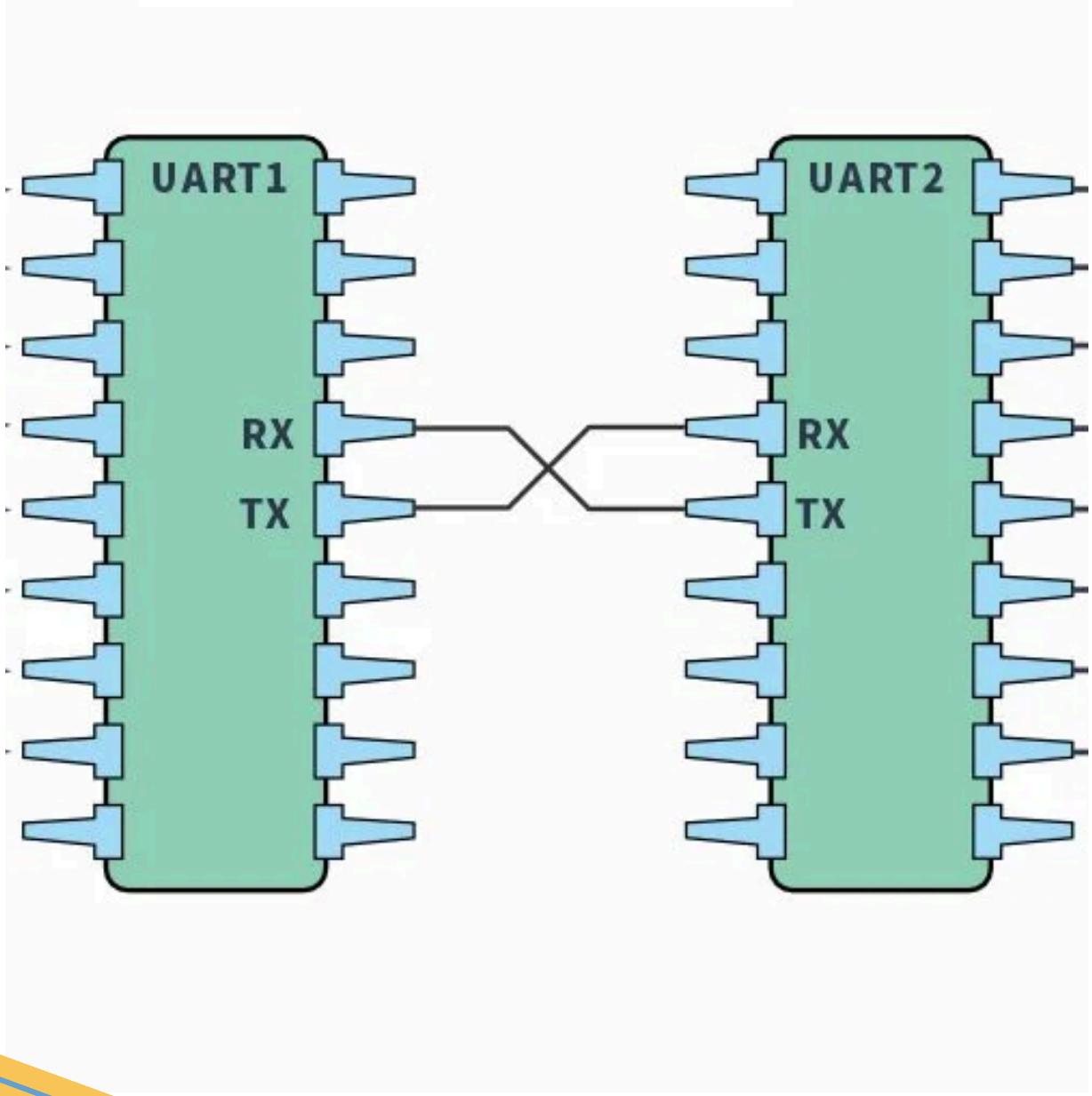
UART

Made by : Salah Waheed



Universal Asynchronous Receiver/Transmitter

(UART)

Table of Contents:

1. Abstract
2. Introduction
3. UART Block Diagram and Components
   A. TX Module
   B. RX Module (x16 oversampling)
   C. Baud Rate Generator
4. Simulation Waves
   A. Test
6. Conclusion

Abstract:

This project presents the design and implementation of a Universal Asynchronous Receiver/Transmitter (UART) using Verilog HDL. The design comprises three modular components: a transmitter (TX) that performs parallel-to-serial conversion, a receiver (RX) that implements 16× oversampling for robust serial-to-parallel conversion, and a configurable baud-rate generator that provides both 1× and 16× timing references.

The RX oversampling improves tolerance to jitter, bit-edge uncertainty, and small baud mismatches, enabling reliable data recovery in noisy or imperfect clocking environments.

Implementation emphasis was placed on clear modular interfaces, synthesizable RTL, and thorough functional verification. A loopback testbench was used during simulation to exercise the full transmit/receive path and validate end-to-end correctness across multiple selectable baud rates.

Results show correct transmission and reception of data bytes under nominal conditions, and the modular approach allows straightforward extension (parity, configurable stop bits, FIFOs) and integration into FPGA projects. The final deliverables include Verilog sources for TX, RX, and the baud generator, a top-level wrapper for integration, and a testbench with waveform recordings for verification.

Introduction:

Universal Asynchronous Receiver/Transmitter (UART) is a fundamental serial communication protocol used extensively for short-range asynchronous links between microcontrollers, sensors, peripherals, and host PCs. UART transmits framed bytes consisting of a start bit, a configurable number of data bits (commonly eight), optional parity, and one or more stop bits. Because UART is asynchronous, both ends must agree on the nominal baud rate, but small differences and noise are common in practical systems. To improve robustness, hardware UART receivers typically employ oversampling — sampling the incoming serial line multiple times per bit period — to reliably detect bit edges and sample in the center of each bit.

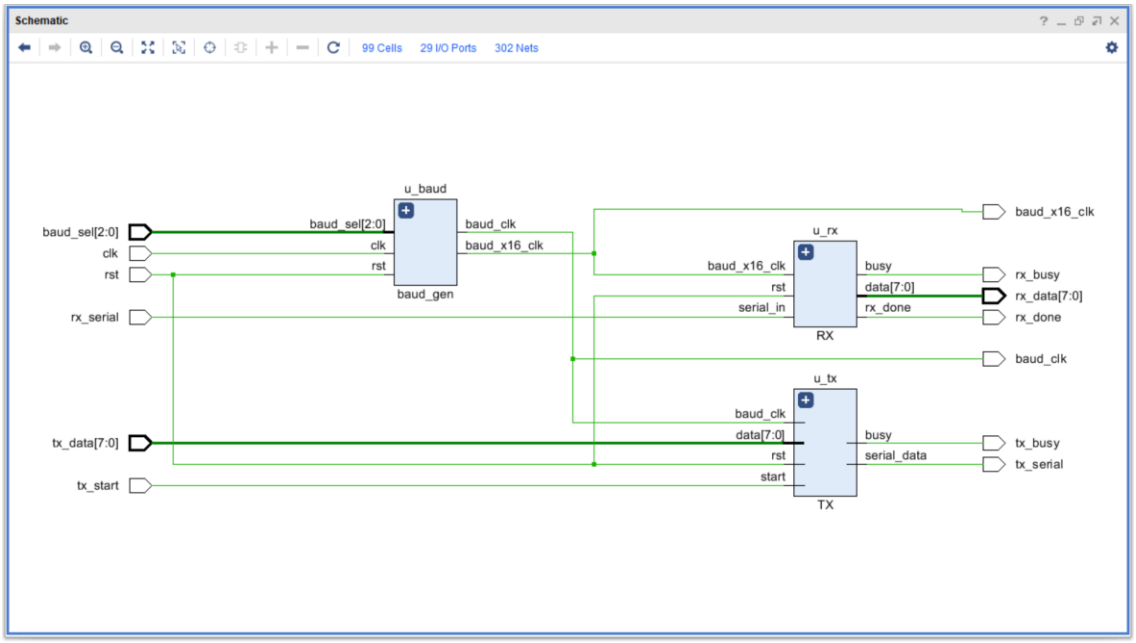
This project implements a compact and modular UART in Verilog with the following key features:

- **Transmitter (TX)**: Serializes an 8-bit parallel input LSB-first, emits a start bit, the data bits, and a stop bit. The TX is driven by a 1× baud clock to guarantee precise bit durations.
- **Receiver (RX)**: Uses a 16× oversampling clock to detect the falling edge of the start bit, wait half a bit time to reach the bit center, and then sample each data bit at its midpoint. This 16× scheme increases tolerance to timing errors and line noise compared to single-sample receivers.
- **Baud Generator**: Provides a selectable set of baud rates and produces both the 1× baud clock for TX and the 16× oversample clock for RX. A compact selection logic supports common rates (9600, 19200, 38400, 57600, 115200), and the design is parameterized by system clock frequency for portability.

The hardware was developed with synthesis in mind (clean FSMs and explicit reset behaviour) and validated using cycle-accurate simulation. A loopback testbench — where the TX output connects to the RX input — was used to verify that transmitted bytes are correctly reconstructed under different baud settings. Waveform captures were analyzed to confirm correct sampling points (middle of bits), proper start/stop detection, and stable rx_done assertions.

UART Block Diagram and Components:

First let's have a look at main components of the circuit:



1. TX Module:

The **Transmitter (TX)** module converts 8-bit parallel input data into a serialized UART frame. Its main responsibilities are:

- **Start bit**: Sends a logic 0 for one bit period to signal the beginning of a frame.
- **Data bits**: Shifts out the 8-bit word **LSB first**, one bit per baud tick.
- **Stop bit**: Sends a logic 1 for one bit period to mark the end of the frame.
- **Clocking**: Operates directly from the **1× baud clock**, ensuring stable bit timing.
- **Status**: Provides a busy flag so external logic knows when transmission is in progress.

This simple FSM ensures reliable serialization and makes the transmitter easy to integrate into larger systems.

2. RX Module (x16 Oversampling):

The **Receiver (RX)** module reconstructs bytes from the incoming serial stream using **16× oversampling**. Its functionality includes:

- **Start bit detection**: Monitors the line for a falling edge and confirms a valid start bit by sampling near the midpoint.
- **Data sampling**: For each bit period, the RX samples on the **8th tick out of 16**, i.e., in the center of the bit, which minimizes timing errors.
- **Stop bit validation**: Ensures the stop bit is high; if not, a framing error can be flagged (extendable).
- **Clocking**: Driven by the **16× baud clock** to achieve oversampling accuracy.
- **Status**: Asserts rx_done when a full byte is received, and busy while receiving.

The oversampling technique improves tolerance against clock mismatches, line noise, and jitter compared to a simple 1× sampling receiver.

3. Baud Rate Generator:

The **Baud Rate Generator** produces timing signals for both TX and RX. It has the following features:

- **Dual clocks**:
- **TX baud clock (1×)** for bit timing in the transmitter.
- **RX oversample clock (16×)** for the receiver's oversampling logic.
- **Configurable rates**: A 3-bit input (baud_sel) selects standard baud rates (e.g., 9600, 19200, 38400, 57600, 115200).
- **Parameterizable**: Designed with a generic system clock frequency, making it portable to different FPGA boards.
- **Implementation**: Uses counters/dividers to derive the required frequencies from the main system clock.

This modular baud generator ensures that the UART design is flexible, reusable, and easy to adapt to different applications.

Simulation Wave:

Waveform simulations were carried out using Vivado. The following was tested:
Loopback test: TX output connected to RX input.
In all cases, RX data matched TX data, confirming correct operation.

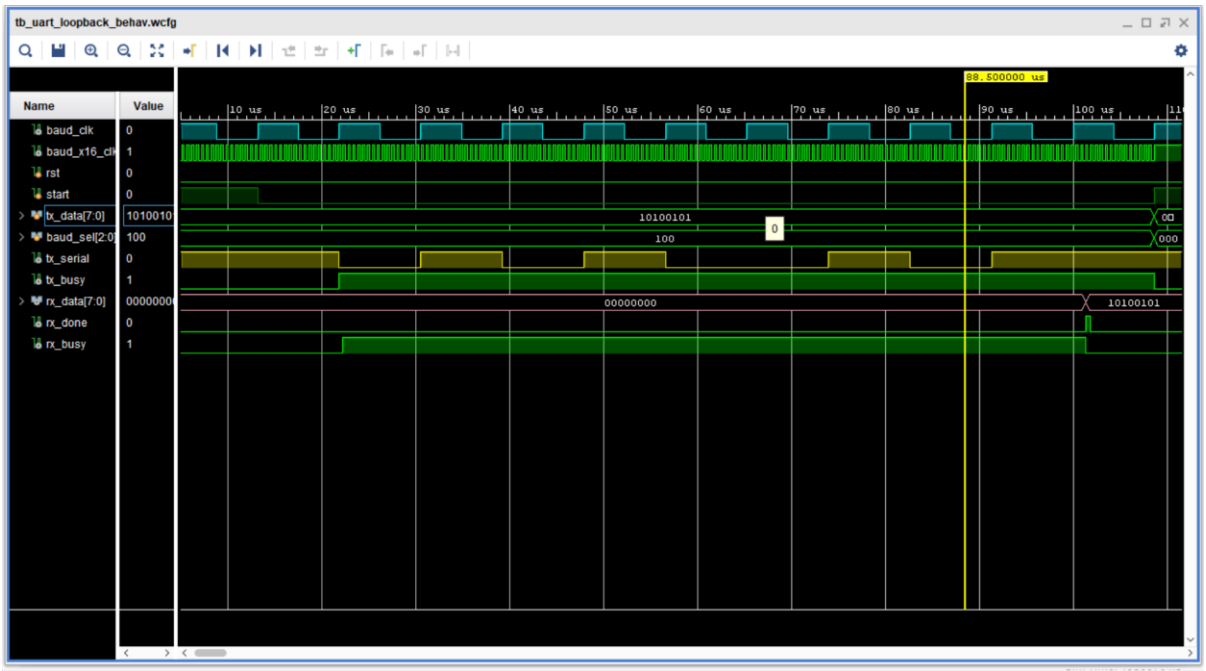


FIGURE 1 : SIMULATION SHOWING TX TRANSMITTING 0XA5 AT 115200 BAUD RATE

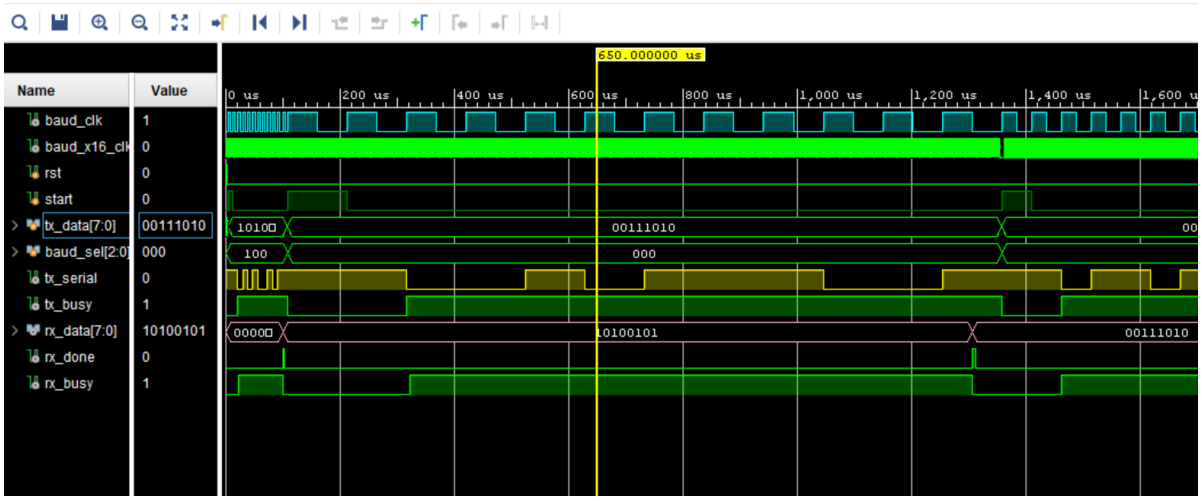


FIGURE 2: SIMULATION SHOWING TX TRANSMITTING 0X 3A AT 9600 BAUD RATE

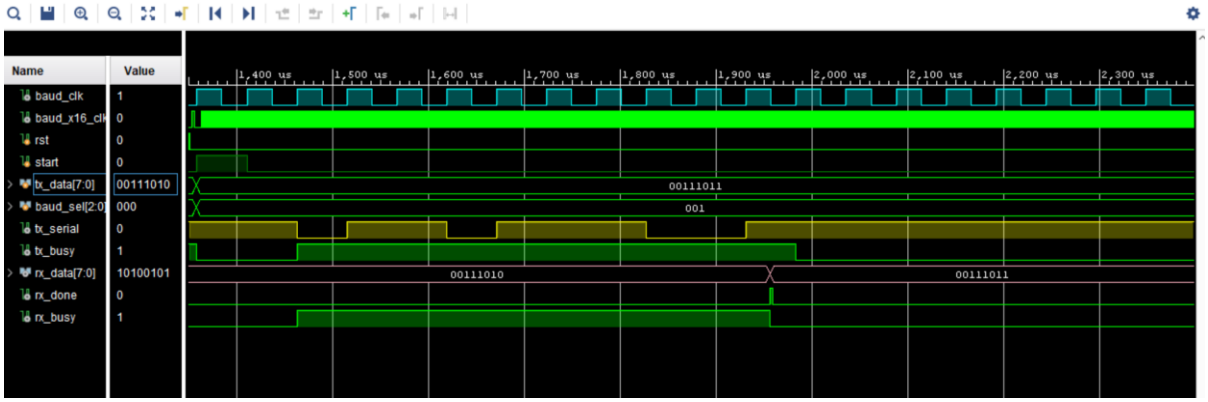


FIGURE 3: SIMULATION SHOWING TX TRANSMITTING 0X 3A AT 19200 BAUD RATE

Conclusion:

This project successfully designed and implemented a UART system with TX, RX, and baud generator modules. The RX module's oversampling method ensures robust data reception. Future work can extend the UART to include parity checking, configurable stop bits, and FIFO buffering.